

1. Bölüm

Temel Bilgisayar Kavramları

1.1 Mühendis ve Teknisyen

Mühendis (**engineer**), ihtiyaç duyulan ürünü; ekonomik (**cost**), güvenli (**safe**) ve kullanışlı (**functional**) olarak zamanında (**on time**) ortaya koyan kişilerdir. Bunu yapmak için; keşif (**invent**), tasarım (**design**), analiz (**analysis**), uygulama (**build**) ve sınamaya (**test**) yaparlar ¹.

Teknisyenler (**technician**) ise genellikle laboratuvarlarda teknik ekipmanlarla ilgilenen veya bu ekipmanlar ile pratik çalışmalar yapan kişilerdir. Genel olarak amacı ekipmanın bakım ve kullanımına ilişkindir.

Konumuz yazılım olduğundan, yazılım mühendisleri istenilen bilgisayar yazılımını ekonomik, güvenli ve kullanışlı olarak zamanında ortaya koyan kişilerdir.

Programcılar ise teknisyenlere karşılık gelir. Yani geliştirme aşamasında bazı yazılım kısımlarının kodlanmasını veya geliştirilen yazılımın çalışır tutulmasını sağlarlar.

1.2 Bilgisayara Niçin İhtiyaç Duyulur?

Bir çiftçi niçin traktöre ihtiyaç duyar? Bir berber niçin elektrikli tıraş makinesi kullanır? Benzer şekilde bir mühendis niçin bilgisayara ihtiyaç duyar? İşte bütün bu soruların cevabı yapılacak işleri daha kısa sürede yapmaktır. Yani bütün araç gereç ve makinalara olan ihtiyacımız işlerimizi **daha hızlı** (**speed**) yapabilmek içindir. Bütün makineler gibi bilgisayar da veriden hızlıca bilgi elde etmemizi ve tekrarlanan hesaplamaları hatasız yapmamızı hızlıca sağlayan makinelerdir.

Veri sorumlusu, veri hazırlama kontrol işletmeni gibi unvanlar bu sebeple ortaya çıkmıştır. Bu unvanlara ihtiyaç duyulmasının sebebi verinin belli bir şekle uygun olarak bilgisayara girilmesi ve işlenen verinin saklanması ve raporlanmasının bilgisayar dünyasında önemli bir yer tutmasıdır. Bilgisayarlar ilk zamanlarda daha çok cebirsel ifadeleri hatasız ve hızlı bir şekilde yapmak için kullanılmışlardır. Bunu daha sonradan ALGOL (**ALGO**rithmic **L**anguage) adını alacak IAL (**I**nternational **A**lgebraic **L**anguage) ya da FORTRAN (**FOR**mula **TRAN**slation) dilinden de anlayabiliriz.

1.3 Veri ve Bilgi

Veriler (**data**), genellikle bir ölçüm ya da sayım sonucu elde edilen ve hakkında az şey bilinen varlıklardır. Örneğin ölçülen hava sıcaklığı, derse giren öğrenci sayısı, tartılan sebzenin ağırlığı gibi şeyler.

Bilgi (**information**) ise verinin işlenerek ortaya çıkan anlamlı verilerdir. Örneğin ortalama hava sıcaklığı, ölçülen yerde bir ölçüm yapmaya gerek kalmadan sıcaklığı yaklaşık olarak bilmeye yarayan bir bilgidir. Bir sınıftaki not ortalaması, o sınıftan rastgele seçilen bir öğrencinin notunu yaklaşık belirleyen bir bilgidir. **Kısacası bilgi, işlenmiş veridir.**

1.4 En Basit Bilgisayar

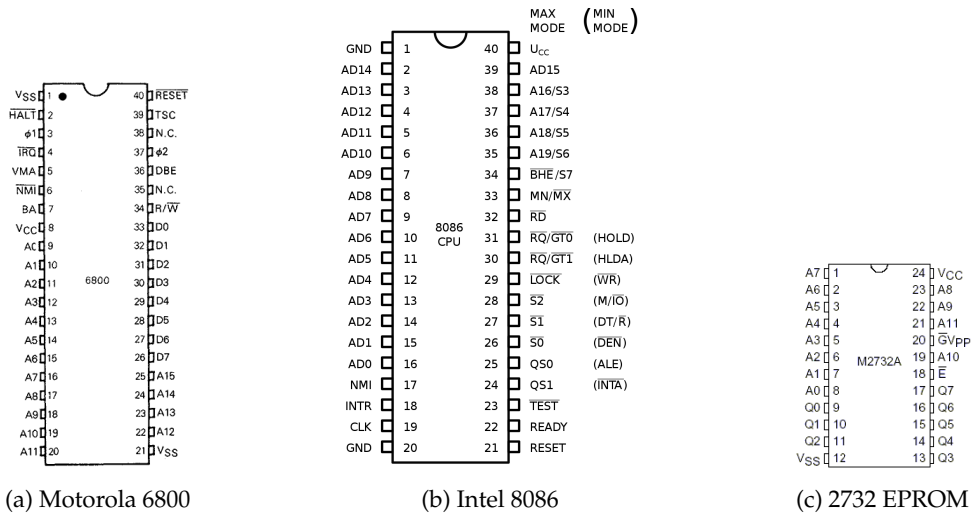
En basit bilgisayar üç bileşenden oluşur;

1. Merkezi İşlem Birimi CPU (**C**entral **P**rocessing **U**nit): Kısaca **işlemci** (**processor**) olarak adlandırılır.
2. Bellek (**memory**): İşlemcinin işleyeceği **emirleri** (**instruction**) ve verileri barındırır.
3. Yol (**bus**): İşlemci ile belleği birbirine bağlayan elektrik yollarıdır.

Hem belleğin hem de işlemcinin adres (**address**) veri (**data**) bacakları bulunur. Bu bacakları karşılıklı olarak yollar (**bus**) birbirine bağlar. Bunların yanında hem işlemcinin hem de belleğin okuma (**read**), yazma (**write**) bacakları bulunur. Ayrıca işlemciye özel kesme (**interrupt**) veya kristal bağlanan bacakları gibi kontrol (**control**) bacakları da bulunur. İşlemci ve bellek, entegre devre (**integrated circuit**) kısaca **yonga** şeklinde

¹www.bls.gov/ooh/architecture-and-engineering

üretirler.



Şekil 1.1: Motorola 6800, Intel 8086 İşlemciler ile 2732 EPROM Belleği

Şekil 1.1’de görüldüğü üzere 6800 işlemcisinin 15 adet adres bacağı, 8 adet veri bacağı bulunmaktadır. 8086 işlemcisinin ise 20 adet adres bacağı bulunmaktadır. Bu bacakların 16 adedi aynı zamanda veri bacağı olarak kullanılmaktadır. İşlemciye karşılık olarak kullanılacak 2732 belleğin ise 12 adet adres bacağı, Q ile gösterilen 8 adet veri bacağı bulunmaktadır.

Görüldüğü üzere işlemci ile bellek arasında bağlantı sağlayan elektrik yolları (bus) adres veya veri taşır. Bu yolların veri taşıyanlarına **veri yolu (data bus)**, adres taşıyanına ise **adres yolu (address bus)** adı verilir. Veri yolu; hem işlemci tarafından belleğe veri yazmak veya bellekten işlemciye taşınacak veriler için kullanıldığından çift yönlüdür. Adres yolu ise belleğin hangi bölgesindeki veriye ulaşılacağını belirleyen hatlar olup tek yönlüdür. 8086 işlemcisinde AD ile işaretli veri ve adres için kullanılan ortak yolların ne zaman adres ne zaman veri göstereceği S0 bacağından belli olur.

Bit

Bilgisayarlar elektrikle çalışan aletlerdir. İşlemci ile bellek arasında bir hatta elektrik varsa 1 yoksa 0 olarak anlamlandırılır. Bunlar **ikili (binary)** sayı sisteminin rakamlarıdır ve kısaca **bit (BInary DiGiT)** olarak adlandırılır. Dolayısıyla; bir elektrik hattının taşıyacağı veri, bir haneli ikili sayı ile, iki elektrik hattının taşıyacağı veri, iki haneli ikili sayı ile ... ifade edilebilir.

İşlemcinin veri yolunun genişliği **kelime uzunluğudur (word length)**. Bu, işlemcinin aynı anda işlem yaptığı bit sayısını da verir. Sekiz (8) bitlik veriye **bayt (byte)** adı verilir ve bellek miktarı genellikle bayt olarak ölçülür.

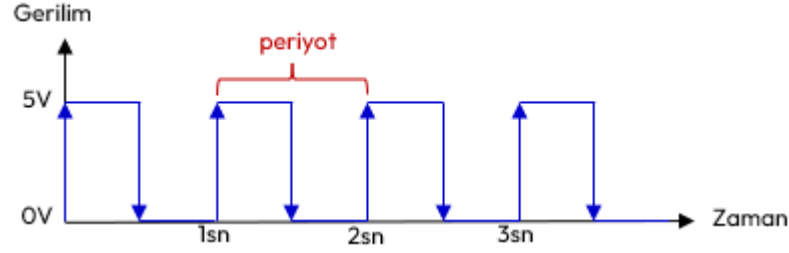
İşlemcinin veri yolu, 8 bit ise BYTE işlemci, 16 bit ise WORD işlemci, 32 bit ise DWORD (Double WORD) işlemci, 64 bit ise QWORD (Quad WORD) işlemci olarak adlandırılır.

İşlemcinin adreslenebilir bellek miktarı ise işlemcinin adres hatlarının sayısı yani adres yolu ile belirlenir. **Adres yoluna konulan her bir adres ile 1 bellek baytı adreslenir.** Adres yolu ne kadar geniş ise adreslenebilir bellek miktarı da o kadar artar. Örneğin 64 bit adres bacağına sahip işlemci $2^{64} = 18.446.744.073.709.551.616$ kadar bellek baytını adresler.

İşlemci	Veri Yolu	Adres Yolu	Adreslenebilir Bellek	Kilo bayt	Mega bayt	Giga bayt
Motorola 6802	8 Bit	16 Bit	$2^{16}=65.536$	64		
Intel 8086	16 Bit	20 Bit	$2^{20}=1.048.576$	1024	1	
Intel 80286	16 Bit	24 Bit	$2^{24}=16.777.216$		16	
Intel Pentium	32 Bit	32 Bit	$2^{32}=4.294.967.296$		4096	4
Intel i7	64 Bit	36 Bit	$2^{36}=68.719.476.736$			64

Tablo 1.1: Bazı İşlemcilerin Adres ve Veri Yolu Genişlikleri

Bilgisayarlarda kablodaki gerilimin değeri genelde 5V, 3.3V, 2.8V, 2V, 1.5V veya 1V gibi değerler alır. **Programcılar için gerilimin ne olduğu değil varlığı önemlidir.** Gerilim değerin yüksek olması fazla akım çekme ve ısınma anlamına gelir. Bu nedenle günümüzde daha düşük gerilimle çalışan işlemciler tasarlanmaktadır.



Şekil 1.2: Frekansı 1 olan Kare Dalga İşareti

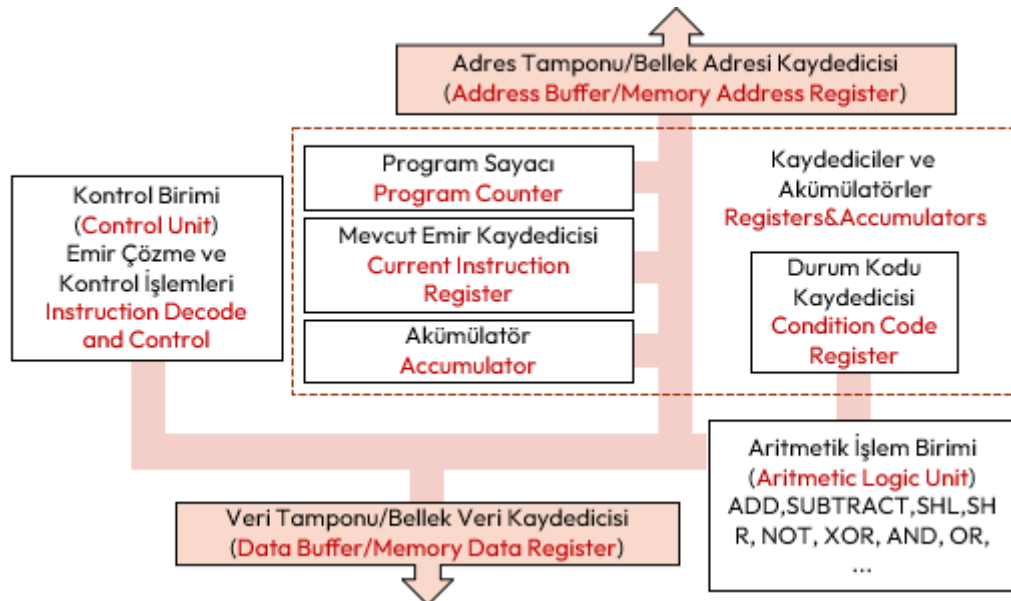
İşlemciler kare dalga olarak oluşturulmuş elektriksel bir işareti **saat (clock)** olarak kullanır. Şekil 1.2’de gösterilen bu elektrik işareti işlemciye ϕ ya da CLK bacaklarından uygulanır. Saniyede 1 kez bir dalga üreten işaretin frekansı $1/1\text{sn} = 1 \text{ Hertz} = 1 \text{ Hz}$. olarak hesaplanır. Bir mikro saniyede bir kez dalga üreten işaretin frekansı; $1/(1\mu\text{s}) = 1/(10^{-6}\text{s}) = 10^6 \text{ Hertz} = 1.000.000 \text{ Hertz} = 1 \text{ MHz}$ olarak hesaplanır.

İşlemciye uygulanan saatin frekansı ne kadar yüksek olursa işlemci o kadar hızlanır. Ancak işlemcilerin de tasarımından ötürü karşılayabileceği belli bir frekans değeri bulunmaktadır. Günümüzde 5 GHz frekansını karşılayan işlemciler bulunmaktadır.

1.5 Emri Alma, Çözme ve İcra Çevrimi

Basit bir işlemciadaki bileşenleri aşağıdaki şekilde açıklayabiliriz;

- ◇ **Kaydediciler (register)**, işlemci içerisinde **çok hızlı e geçici adres veya veri saklayan** alanlardır.
- ◇ **Akümülatörler (accumulator)** ise bellekten alınan verileri saklamak veya işlenen sonuçları saklamak için en sık kullanılan kaydedicilerdir.
- ◇ **Program sayacı PC (program counter)**, işlemcinin icra edeceği **emri (instruction)** takip için kullanılır. Getirilecek bir sonraki emrin bellek adresini içerir.
- ◇ **Mevcut Emir kaydedicisi CIR (current instruction register)** işlemcinin o an icra ettiği emirdir.
- ◇ **Koşul kodu kaydedicisi (condition code register)**, herhangi bir işlemin durumunu gösteren farklı bayraklar (**flag**) içeren kaydedicidir.
- ◇ **Aritmetik İşlem Birimi ALU (arithmetic logic unit)**, işlemcinin toplama çıkarma ve karşılaştırma yapan birimidir.
- ◇ **Kontrol Birimi CU (control unit)** Orkestra şefi gibidir, emirleri çözer ve yorumlar ve gerekli bileşenlere sinyal gönderir.
- ◇ **Veri Yolları (bus)**, verinin bileşenler arasında taşındığı birden çok elektrik hattıdır.



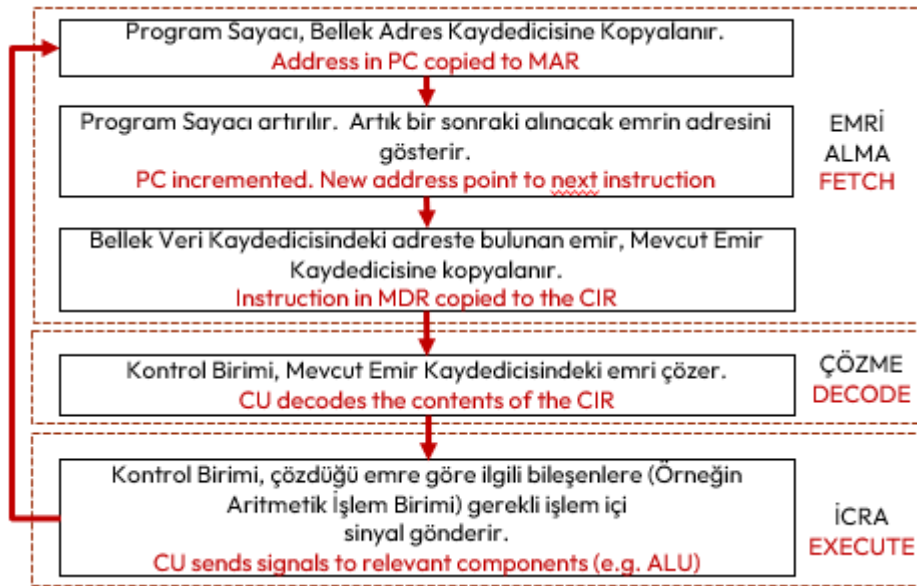
Şekil 1.3: Basit Bir İşlemcinin Yapısı

Bu işlemci tasarımı aynı zamanda **depolanmış program (stored program)** kavramı olarak adlandırılır ve **Von Neumann Mimarisi** olarak da bilinir. *John Von Neumann* tarafından 1946 ila 1949 yılları arasında tasarlanan ve Şekil 1.3’de verilen bu mimari günümüzde modern bilgisayarlarda hala kullanılmaktadır.

Basit bir bilgisayara elektrik verildiğinde, işlemci aşağıdaki üç adımlık **emir alma (fetch)**, **çözme (decode)**

ve **icra** (**execute**) çevrimini elektrik kesilene kadar sürekli tekrarlanır². Bu çevrime **emir çevrimi** (**instruction cycle**) adı da verilir.

1. **Emri Alma** (**fetch**): Bu aşamada program sayacının (**program counter**) gösterdiği adresteki emir (**instruction**) bellekten okunur:
 - (a) Program sayacı , bir sonraki emrin adresini tutar.
 - (b) Bu adres **bellek adres kaydedicisine** (**memory address register**) kopyalanır.
 - (c) Talimat bellekten çağrılır ve **bellek veri kaydedicisine** (**memory data register**), oradan da **mevcut emir kaydedicisine** (**current instruction register**) aktarılır.
 - (d) Program Sayacı, bir sonraki emri işaret etmek için bir artırılır.
2. **Çözme** (**decode**): Kontrol birimi (control unit) tarafından emrin ne anlama geldiği ve buna bağlı nelerin yapılacağını (örneğin ulaşılacak bellek adresi değiştirilir) belirler:
 - (a) Mevcut emir kodu (**current instruction code**), ne yapılacağını belirten kısım yani işlem kodu (**opcode**) ve hangi veriyle yapılacağını belirten işlenen (**operand**) kısımdan oluşur.
 - (b) Kontrol birimi, varsa emrin işaret ettiği veriyle işlemin gerçekleşmesi için gerekli donanım parçalarını hazırlar.
3. **İcra** (**execute**): Çözülen emir koduna bağlı olarak işlemcinin bazı bileşenleri çalıştırılır:
 - (a) Eğer bir matematiksel işlem veya mantıksal karşılaştırma gerekiyorsa, bu görev aritmetik işlem birimini ALU (**Aritmetic Logic Unit**) tarafından yapılır. İşlem sonuçları ortaya çıkan değerler ise akümülatörlere (**accumulator**) veya bellek veri kaydedicisi (**memory data register**) üzerinden belleğe yazılır.
 - (b) Bazı durumlarda ise kaydedici (**register**) içerikleri değiştirilir.



Şekil 1.4: İşlemcinin Emri Alma, Çözme ve İcra Çevrimi

Burada çözülecek emirlerin (**instruction**) her biri bir ikili sayıya karşılık gelir. Bu sayılara **emir kodu** (**instruction code**), **makine kodu** (**machine code**) veya **işlem kodu** (**opcode**) olarak da bilinirler. İşlemciler tüm sayıları emir kodu olarak anlamaz. Yani sınırlı sayıda sayı emri tanır. Tablo 1.1'de verilen Motorola 6802 işlemcisi 72 farklı emri, Intel 8086 işlemcisi ise 116 farklı emri tanır. Çözülebilecek tüm emirlerin oluşturduğu kümeye **emir seti** (**instruction set**) adı verilir.

1.6 İşlemci ve Bellek İş Birliğinin Temeli

Bir bilgisayarın çalışma mantığını anlamak için iki temel soruya yanıt vermemiz gerekir: "Veri nerede durur?" ve "İşlem nerede yapılır?"

Ş1.6.1 İşlem Nerede Yapılır?

İşlemci yani CPU (Central Processing Unit), bilgisayarın "beyni" veya bir mutfağın "şefi" gibidir. Yazdığınız kodlardaki tüm matematiksel hesaplamalar, mantıksal karşılaştırmalar ve karar verme süreçleri burada gerçekleşir. Ancak işlemcinin kendi içinde veri saklama kapasitesi çok sınırlıdır; o sadece önüne gelen işi o an yapmakla görevlidir.

²<https://www.youtube.com/watch?v=ByllwN8q2ss>

§1.6.2 Veri Nerede Durur?

Bellek RAM (Random Access Memory), bilgisayarın **çalışma masası**'dır. Programınız çalışırken kullandığınız değişkenler, diziler ve nesneler geçici olarak burada tutulur. İşlemcinin bir veriyi işlemesi için, o verinin önce bellekten çağırılması gerekir. Bellek hızlıdır ancak "geçicidir"; bilgisayar kapandığında içindeki tüm veriler silinir.

Bu ilişkiyi daha somutlaştırmak için bir mutfağı hayal edelim:

- ◊ İşlemci (CPU): Yemek yapan aşçıdır. İşlemi (doğrama, pişirme) o yapar.
- ◊ Bellek (RAM): Aşçının önündeki kesme tahtası veya tezgâhtır. O an kullanılan malzemeler (veriler) burada durur.
- ◊ Sabit Disk (HDD/SSD): Mutfağın kileri veya buzdolabı gibidir. Tüm malzemeler orada saklanır (kalıcı depolama).

Programınızı çalıştırdığınızda, verileriniz kalıcı depolamadan (disk) alınır ve belleğe (RAM) yerleşir. İşlemci (CPU) ise bellekte hazır bekleyen bu verileri sırayla alır, işler ve sonucu tekrar belleğe yazar.

1.7 İşlemci Tasarlama Stratejileri

İşlemciler, aşağıda verilen üç strateji ile tasarlanırlar;

1. **İndirgenmiş Emir Setli İşlemciler** olan RISC (**Reduced Instruction Set Computing**) İşlemci: Az sayıda emir alan yüksek hızlı işlemci tasarımı. RISC işlemciler az sayıda emre sahip olduğundan emrin çözülmesi ve icrası da kısa zaman almaktadır. Bu da çalıştırılacak kodu büyütür.
2. **Karmaşık Emir Setli İşlemciler** olan CISC (**Complex Instruction Set Computing**) İşlemci: Çok fazla emri anlayan karmaşık nispeten yavaş işlemci tasarımı. CISC işlemciler, çok sayıda emir tanıyabildiğinden emrin çözülmesi ve icra edilmesi nispeten daha uzun sürer. Ancak yüksek düzey dillerin daha kolay makine koduna çevrilmesini kolaylaştırır.
3. **Özel İşlemci**: Bunlar grafik kartları ve yapay zekâ gibi belli amaçlar için tasarlanmış işlemcilerdir.

Genel olarak işlemcilerde emir çevrimi (**instruction cycle**) 1 saat periyodunda tamamlanmaz. RISC işlemciler için bu çevrim, birçok emir için 1 saat periyodunda tamamlanır. CISC işlemciler için ise birden fazla saat periyodu gerekir. Çünkü CISC işlemcilerin emir seti daha geniş olduğundan emrin çözülmesi daha fazla vakit alır. Bu nedenle RISC işlemciler CISC işlemcilere göreceli olarak daha hızlıdır.

Günümüzde ticari olarak satılan işlemcilerin çoğu ortak emir seti kullanmaya başlamışlardır;

- ◊ Masaüstü bilgisayarların birçoğu Intel ve AMD Marka işlemciler kullanır ve bunlar IA-16, IA-32, x86-16, x86-64, AMD64, ... emir setleri ve eklentilerini desteklemektedir.
- ◊ Bilinen birçok telefon işlemcisi Acorn Risc Machine-ARM tabanlı işlemciler kullanır. Bu işlemcilerde ARMv7, ARMv8, ... emir setleri ve eklentileri kullanılmaktadır.

1.8 Onaltılı Sayı Sisteminin Kullanılması

İşlemci ile bellek arasında verilerin elektrik yolları (**bus**) ile taşındığı anlatılmıştı. Veri taşıyan elektrik hattı sayısı işlemci ile bellek arasında taşınabilecek verinin sınırlarını da belirler. Bir elektrik hattının taşıyacağı veri bir bit ile (0 veya 1) gösterilebilir. İki elektrik hattının taşıyacağı veri ise iki bit ile (00, 01, 10 veya 11) 2^2 kadar farklı gösterilebilir. Benzer şekilde n adet elektrik hattının taşıyacağı veri 2^n farklı alternatifte veri olacaktır.

Bit sayısı arttıkça ikili sayının okunup yazılması da çok zor olacaktır. Günümüzde 64 bitlik hatta 1024 bitlik veri yoluna sahip bilgisayarlar bulunmaktadır. Bunun üstesinden gelmek için (**hexadecimal**) sayı sistemi kullanılır. Çünkü her 4 elektrik hattının verisi onaltılık sayı sisteminin bir rakamına karşılık gelir.

Onaltılı	İkili	Onlu	Onaltılı	İkili	Onlu
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

Tablo 1.2: Onaltılı Sayı Rakamları ve İkili ve Onlu Karşılıkları

Kod İçerisinde Onaltılı Sayılar

Kod içerisinde onaltılık sayıların rakamlarının diğer sayı sistemindeki rakamlarla karışmaması için sonuna **h** ya da **H** konularak **113AH** şeklinde veya çoğunlukla başına **0x** konularak **0x1133** şeklinde kullanılır.

Örnek olarak ikili tabanda verilen 64 bitlik **0001 1010 1011 1101 0101 0100 1001 1010 0011 0010 0100 0110 1001 1101 1100 1111** sayısının onaltılı tabanda karşılığı oldukça kısa (16 hane) olarak **1ABD 549A 3246 9DCF** şeklindedir.

Onaltılı Sayılar

Onaltılı sayının **A,B,C,D,E** ve **F** rakamları büyük harf olacağı gibi küçük harf de olabilir. Anlam olarak bir şey değişmez.

Bilgisayar dünyasında ikili sayıları onaltılı sayı sistemiyle ifade etmek çoğunlukla zorunluluktur. Zamanla standart hale gelmiştir.

1.9 Program ve Programlama

Belli bir işlemi gerçekleştirmek üzere, işlemciye verilen bir dizi emre (instruction), **bilgisayar programı** (**computer program**) denir. Bu bir dizi emrin, işlemcinin çalıştıracağı şekilde hazırlanması işlemine **programlama** (**programming**) denir. Programlama **emir kodları** (**instruction code**) ile yapıldığından bu programın dili, **makine dili** (**machine language**) olarak adlandırılır³.

Her işlemci üreticisi kullandığı emir seti farklı olduğundan yani her bir emre farklı makine kodu verdiğinden programlama oldukça zorlu ve teknik bilgi gerektirmekteydi. Burada daha önce yazılmış bir programın ne yaptığını anlamak için her defasında bu listelere defalarca bakmak gerekiyordu. Programın okunurluğunu artırmak için her bir emre, **hatırlatıcı isimler** (**mnemonic**) verilmeye başlandı. Tablo 1.3'de 6800 işlemcisinin hatırlatıcı isim listesi verilmiştir.

Montajlayıcı

Hatırlatıcı isimlerle yazılan programları emir kodlarına (**instruction code**) çeviren programlara **montajlayıcı** (**assembler**) adı verilir. Bu programlar, hatırlatıcı isimlerle yazılmış program kodlarını makine koduna çeviren ilk **derleyicilerdir** (**compiler**). Derleyici terimi genelde, ileride bahsedeceğimiz C gibi yüksek seviyeli dilden makine/assembly koduna çeviren programlar için kullanılır.

Bu hatırlatıcı isimlerle yazılan program, **montaj kodu** (**assembly code**) olarak adlandırılır. Montaj kodları bir metin dosyasına yazılarak bir dosyaya saklanır. Bu dosyalar montaj dosyaları olarak adlandırılır.

Aşağıda x86 emir seti kullanan Linux işletim sisteminde örnek montaj kodu örneği verilmiştir; Örnekte, genel amaçlı **CL** kaydedicisine koyulan sayıya kadar 2 ve 3'e bölünebilen sayıların toplamını bulup yine genel amaçlı **DX** kaydedicisine konulmaktadır⁴.

```
section .text ; programı oluşturan emir kodları buradan başlar
global _start
_start:
    mov dx, 0 ; dx 0 olsun. Toplamı tutacaktır.
    mov cl, 99 ; 99 kez tekrarlanacak.

sum:
    mov ax, cx ; cx, ax e kopyalanır. Bölme işlemi için.
    mov bl, 3 ; bl 3 olsun.
    div bl ; ax/bl.
    cmp ah, 0 ; Kalan olup olmadığı kontrol ediliyor.
    jne cont ; Kalan yoksa, 'cont' etiketine git.
    mov bl, 2 ; Sonra bl 2 olsun.
    div bl ; ax/bl
    cmp ah, 0 ; Kalan olup olmadığı kontrol ediliyor.
    jne cont ; Kalan yoksa, 'cont' etiketine git.
```

³ISO/IEC 2382:2015. ISO. 2020-09-03. [Software includes] all or part of the programs, procedures, rules, and associated documentation of an information processing system

⁴<https://github.com/armut/x86-Linux-Assembly-Examples>

Mnemonic	Emir Kodu	Açıklama
LDAA	96	LOAD M to A ($M \rightarrow A$, Direct)
LDAB	D6	LOAD M to B ($M \rightarrow B$, Direct)
STAA	97	STORE A to M ($A \rightarrow M$, Direct)
STAB	D7	STORE B to M ($B \rightarrow M$, Direct)
TAB	16	TRANSFER A to B ($A \rightarrow B$)
TBA	17	TRANSFER B to A ($B \rightarrow A$)
CLR	7F	CLEAR Memory ($00 \rightarrow M$, Extended)
CLRA	4F	CLEAR A ($00 \rightarrow A$)
CLRB	5F	CLEAR B ($00 \rightarrow B$)
ABA	1B	ADD A to B ($A + B \rightarrow A$)
SBA	10	SUBTRACT B from A ($A - B \rightarrow A$)
ADDA	9B	ADD M to A ($A + M \rightarrow A$, Direct)
ADDB	DB	ADD M to B ($B + M \rightarrow B$, Direct)
SUBA	90	SUBTRACT M from A ($A - M \rightarrow A$, Direct)
SUBB	D0	SUBTRACT M from B ($B - M \rightarrow B$, Direct)
INCA	4C	INCREMENT A ($A + 1 \rightarrow A$)
INCB	5C	INCREMENT B ($B + 1 \rightarrow B$)
DECA	4A	DECREMENT A ($A - 1 \rightarrow A$)
DECB	5A	DECREMENT B ($B - 1 \rightarrow B$)
DAA	19	DECIMAL ADJUST A (BCD Formatlaması)
ANDA	94	LOGICAL AND M with A ($A \cdot M \rightarrow A$, Direct)
ANDB	D4	LOGICAL AND M with B ($B \cdot M \rightarrow B$, Direct)
ORAA	9A	LOGICAL INCLUSIVE OR M with A ($A + M \rightarrow A$, Direct)
ORAB	DA	LOGICAL INCLUSIVE OR M with B ($B + M \rightarrow B$, Direct)
EORA	98	EXCLUSIVE OR M with A ($A \oplus M \rightarrow A$, Direct)
COMA	43	COMPLEMENT A (A'nın 1'e tümleneni $\rightarrow A$)
PSHA	36	PUSH A onto Stack ($A \rightarrow \text{Stack}$)
PULA	32	PULL A from Stack ($\text{Stack} \rightarrow A$)
PSHB	37	PUSH B onto Stack ($B \rightarrow \text{Stack}$)
PULB	33	PULL B from Stack ($\text{Stack} \rightarrow B$)
DES	34	DECREMENT Stack Pointer ($SP - 1 \rightarrow SP$)
INS	31	INCREMENT Stack Pointer ($SP + 1 \rightarrow SP$)
JMP	7E	JUMP ($PC \leftarrow \text{Adres}$, Extended)
JSR	BD	JUMP TO SUBROUTINE (Alt programa dallanma)
RTS	39	RETURN FROM SUBROUTINE (Alt programdan dönüş)
BRA	20	BRANCH ALWAYS (Koşulsuz dallanma, Relative)
BEQ	27	BRANCH IF EQUAL ($Z=1$ ise dallan, Relative)
BNE	26	BRANCH IF NOT EQUAL ($Z=0$ ise dallan, Relative)

Tablo 1.3: 6800 Emir Seti

```

add    dx, c    ; Değilse, cx deki sayı 2 ve 3'e bölünür. Toplama ekle.
cont:
loop   sum      ; Bir sonraki sayı için 'sum' etiketine dön.
exit:
mov    eax, 1   ; Artık programdan çıkıyoruz.
mov    ebx, 0   ; Return 0.
int    80h      ; linux çekirdeğini çağır. (soft interrupt 80h!).

```

1.10 Program Türleri

Geliştirdiğimiz programlar üç ana grupta incelenebilir;

- ♦ **Sistem yazılımları**, bilgisayar donanımını yöneten ve uygulama yazılımları için bir platform sunan temel katmandır; işletim sistemleri (Windows, Linux) ve sürücüler bu grubun en net örnekleridir.
- ♦ **Uygulama yazılımları**, son kullanıcının belirli görevleri (metin yazma, internette gezinme, oyun oynama) yerine getirmesi için tasarlanmış, donanımdan ziyade kullanıcı odaklı araçlardır.
- ♦ **Gömülü yazılımlar** ise genel amaçlı bilgisayarlardan ziyade; beyaz eşyalar, otomobiller veya endüstriyel robotlar gibi belirli bir donanımı kontrol etmek üzere o cihazın içine hapsedilmiş, genellikle kısıtlı kaynakla çalışan ve yüksek kararlılık gerektiren özel kod parçalarıdır.

Aşağıdaki tabloda farkları yer almaktadır;

Özellik	Sistem Yazılımları	Uygulama Yazılımları	Gömülü Yazılımlar
Ana Hedef	Donanımı yönetmek	Kullanıcıya hizmet etmek	Cihazı çalıştırmak
Kullanıcı Etkileşimi	Arka planda çalışır	Doğrudan etkileşim kurar	Genellikle görünmezdir
Donanım Bağımlılığı	Çok yüksek	Düşük (Platform bağımlı)	Tam bağımlı
Örnek	Windows 11, macOS	Microsoft Word, Chrome	Çamaşır makinesi kontrolcüsü

Tablo 1.4: Yazılımlar ve Farkları

1.11 Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri

Genel amaçlı yani herkesin kendi amacına göre kullanacağı bilgisayarlar, veri girişini sağlayan klavye, ve-rileri görüntüleneceği monitör, işlenen verilerin elektrik kesildiğinde saklanacağı disk ve benzeri donanımlara sahip olmalıdır.

Genel amaçlı bilgisayarın kullanıcı isteklerine cevap verebilmesi için;

1. Çeşitli programlara sahiptirler. İşte bu programlar bütününe işletim sistemi (**operating system**) adı verilir. DOS, Windows, Unix, Linux, MacOS, ChromeOS, OS/2 bunların en çok bilinenleridir.
2. Elektrik verildiğinde klavye, monitör ve disk olup olmadığı kontrol eden **BIOS (Basic Input Output System)** programı çalıştırılır. BIOS gerekli kontrollerden sonra bilgisayara işletim sistemini yükler. İşletim sistemi yüklendikten sonra, işletim sistemi kullanıcının vereceği komutları bekler.
3. Kullanıcı işletim sistemine vereceği komutlarla yeni bir program derleyebilir, derlenmiş bir programı çalıştırabilir.
4. Kullanıcının vereceği komutlar grafik olmayan işletim sistemlerinde (DOS, Unix, Linux gibi) komut yazılarak verilir. Komut yazılan bu ekrana konsol, terminal ya da komut satırı adı verilir. Sunucu işletim sistemlerinde hala tercih edilmektedir.
5. Grafik ara yüzlerine sahip işletim sistemlerinde (Windows ve MacOS İşletim Sistemleri ile KDE, Mate, GNome, Cinnamon, LXQt, XFce ve Deepin gibi grafik ara yüzleri içeren Linux İşletim Sistemleri) fare tıklamasıyla çoğu komut verilebilmektedir.

İşletim sistemlerinin yaygınlaşması ile kullanıcının bilgisayarı donanım bilmeden yapabilmesi sağlanmışır.

İşletim Sistemi Dili

Günümüzdeki işletim sistemlerinin birkaçı hariç tamamında temel programlama dili olarak C dili kullanılmaktadır.

1.12 Düşün ve Çöz

- ◊ **Düşün:** Bir mühendis ile teknisyen arasındaki temel farkı, bir yazılım projesindeki görev dağılımı üzerinden örneklendiriniz.
- ◊ **Düşün:** CPU içindeki kaydedicilerin (**register**) hız ve kapasite açısından bellekten farkı nedir? Neden tüm verilerimizi kaydedicilerde tutmuyoruz?
- ◊ **Düşün:** *Von Neumann* mimarisinde veri ve komutların aynı yolu (bus) kullanmasının **darboğaz (bottle-neck)** yaratma riskini tartışınız.
- ◊ **Çöz:** 32-bit veri yoluna ve 24-bit adres yoluna sahip bir işlemcinin adresleyebileceği maksimum bellek miktarını MB cinsinden hesaplayınız.
- ◊ **Çöz:** 16-bit adres yoluna sahip bir mikrodenetleyicinin adresleyebileceği maksimum bellek miktarını KB cinsinden hesaplayınız.
- ◊ **Çöz:** Bir bilgisayarın 8 GB RAM'i tam olarak adresleyebilmesi için adres yolunun (**address bus**) en az kaç bit olması gerektiğini bulunuz.